SPRING DATA JPA – 50 Deep Interview Questions (3-Line Answers)

1. What happens internally when save() is called?

When save() is called, the entity becomes managed inside the Persistence Context.

If it's new, INSERT is scheduled; if existing, UPDATE is scheduled.

Actual SQL executes during flush or transaction commit.

2. What is Persistence Context?

Persistence Context is a first-level cache maintained by EntityManager.

It stores managed entities and ensures entity identity.

Prevents duplicate queries for same entity in one transaction.

3. What is Dirty Checking?

Dirty checking automatically detects changes in managed entities.

During transaction commit, JPA compares object state with snapshot.

If changed, it generates UPDATE queries automatically.

4. What is First-Level Cache?

It is session-level cache inside Persistence Context.

Enabled by default in JPA.

Works only within a single transaction.

5. What is Second-Level Cache?

Second-level cache is shared across multiple sessions.

It reduces repeated database hits.

Requires additional configuration like EhCache.

6. What is the N+1 Problem?

Occurs when fetching parent entity and lazy children in loop.

Generates one query for parent and N for children.

Causes serious performance degradation.

7. How to solve N+1 problem?

Use JOIN FETCH in JPQL queries.

Use @EntityGraph to fetch associations efficiently.

Avoid accessing lazy fields inside loops.

8. What is JPQL?

JPQL is object-oriented query language.

It works on entity objects instead of database tables.

Converted internally into SQL by Hibernate.

9. Native Query vs JPQL?

Native query uses raw SQL syntax.

JPQL uses entity names and object fields.

Native queries provide DB-specific optimizations.

10. What is @Transactional?

Marks method as transactional.

Ensures atomicity of operations.

Rollback happens automatically on RuntimeException.

11. What is Transaction Propagation?

Defines how transactions behave when calling another transactional method.

Examples include REQUIRED and REQUIRES_NEW.

Controls transaction boundaries.

12. What is Isolation Level?

Isolation defines visibility of data between transactions.

Levels include READ_COMMITTED and SERIALIZABLE.

Prevents dirty reads and phantom reads.

13. What is Flush?

Flush synchronizes Persistence Context with database.

Executes pending SQL statements.

Happens automatically before commit.

14. What is Clear?

Clear detaches all managed entities from context.

Removes them from first-level cache.

Useful in batch processing.

15. What is Entity Lifecycle?

States include Transient, Persistent, Detached, and Removed.

Entity transitions between states based on operations.

Managed entities are tracked by Persistence Context.

16. What is CascadeType?

Defines propagation of operations to child entities.

Types include PERSIST, MERGE, REMOVE.

Used in entity relationships.

17. What is orphanRemoval?

Deletes child entity if removed from parent collection.

Ensures database consistency.

Used with OneToMany relationships.

18. What is Lazy Loading?

Association loads only when accessed.

Improves performance by reducing unnecessary queries.

Default for OneToMany relationships.

19. What is Eager Loading?

Association loads immediately with parent entity.

Can cause performance issues if not needed.

Default for ManyToOne.

20. What is Optimistic Locking?

Uses version field to detect concurrent updates.

Prevents lost updates.

Does not lock database rows.

21. What is Pessimistic Locking?

Locks database row during transaction.

Prevents concurrent access.

Used in high-contention scenarios.

22. What is @Version annotation?

Used for optimistic locking.

Adds version column in table.

Increments automatically on update.

23. What is Fetch Join?

Fetch join retrieves related entities in single query.

Avoids N+1 problem.

Used in JPQL queries.

24. What is Batch Processing?

Used for handling large datasets efficiently.

Flush and clear periodically.

Improves memory usage.

25. What is EntityGraph?

Defines fetch plan dynamically.

Improves performance by controlling loading strategy.

Avoids unnecessary eager loading.

26. What is @Modifying?

Used with @Query for update/delete operations.

Required for non-select queries.

Must be used inside transaction.

27. What is Query Hint?

Provides optimization hints to JPA provider.

Improves performance in specific scenarios.

Used for caching or fetch size.

28. What is Projection?

Fetches partial fields instead of full entity.

Improves performance.

Can be interface-based or DTO-based.

29. What is DTO Projection?

Maps query result to custom object.

Reduces unnecessary data loading.

Improves API performance.

30. What is Derived Query Method?

Spring Data generates query from method name.

Example: findByEmailAndStatus.

Reduces boilerplate code.

31. What is Composite Key?

Primary key made of multiple columns.

Implemented using @EmbeddedId or @IdClass.

Requires equals and hashCode methods.

32. What is @Embeddable?

Defines reusable embedded object.

Used inside entity.

Maps multiple columns as single object.

33. What is ManyToMany mapping issue?

Creates join table automatically.

Can cause performance overhead.

Better to convert into two OneToMany mappings.

34. What is Transaction Rollback?

Rollback occurs on RuntimeException.

Prevents partial updates.

Can be customized using rollbackFor.

35. What is Entity Detachment?

Entity becomes detached when session closes.

Changes are not tracked after detachment.

Must merge to persist again.

36. What is merge()?

Copies detached entity state into managed entity.

Returns managed instance.

Useful for updates.

37. What is persist()?

Makes transient entity managed.

Schedules INSERT operation.

Fails if entity already exists.

38. What is remove()?

Deletes entity from database.

Entity must be managed.

Schedules DELETE query.

39. What is LazyInitializationException?

Occurs when accessing lazy entity outside transaction.

Happens if session is closed.

Solved using fetch join or DTO projection.

40. What is Open Session in View?

Keeps session open during web request.

Allows lazy loading in view layer.

Can cause performance issues.

41. What is @OrderBy?

Sorts collection at database level.

Defined in entity mapping.

Improves consistent ordering.

42. What is LockModeType?

Defines locking behavior in queries.

Includes OPTIMISTIC and PESSIMISTIC_WRITE.

Controls concurrency.

43. What is saveAndFlush()?

Saves entity and flushes immediately.

Forces SQL execution.

Useful in testing scenarios.

44. What is Performance Tuning in JPA?

Use pagination and projections.

Avoid N+1 queries.

Use proper indexing in DB.

45. What is Indexing impact?

Indexes improve query speed.

Should be added on frequently searched columns.

Improves performance significantly.

46. What is Query Timeout?

Limits execution time of query.

Prevents long-running queries.

Configured via query hints.

47. What is Bulk Update issue?

Bulk update bypasses Persistence Context.

Managed entities may become inconsistent.

Requires clear() after execution.

48. What is @EntityListeners?

Triggers lifecycle callbacks.

Used for auditing.

Executes before or after persistence events.

49. What is Auditing?

Tracks createdBy and modifiedDate fields.

Enabled using @EnableJpaAuditing.

Useful in enterprise apps.

50. How to optimize large dataset queries?

Use pagination and streaming.

Avoid loading full entity graphs.

Use projections for better performance.